

AsymptoticInverse 1.8.0

Correctness, representation, performance,
and the Wolfram Language boundary

Prepared for Vladimir Reshetnikov

9 September 2026

Repository	https://github.com/VladimirReshetnikov/Asymptotic
Audited revision	07a9781212beb2eeb9ff16aa625b50ac27974078
Package version	1.8.0
Native audit runtime	Wolfram Language 15.0.0, Linux x86-64
Evidence	Pinned source inspection, selected native evaluations, independent exact checks

Principal conclusion. The package is a substantial, useful precision-aware layer around a specialized asymptotic algebra, not merely a wrapper for **Series**. Its strongest contributions are ordered irrational power-log inversion, explicit error transport, specialized inverse scales, and retained computational provenance. The immediate priority is to close four boundary defects: inconsistent real-power checks, eager dense export, loss of logarithmic error information during export, and incomplete capture of ambient assumptions.

Abstract

This review examines the implementation and engineering contracts of AsymptoticInverse at a fixed September 2026 revision. It compares the package with Wolfram Language’s existing asymptotic, series, inversion, and composition facilities; derives the mathematical invariants that its sparse jets must preserve; and supplies concrete counterexamples and targeted repairs. Three semantic problems were reproduced in a native Wolfram kernel: a nested fractional-power path accepts a remainder without a real-branch proof, conversion to `SeriesData` forgets a logarithmic remainder factor, and an expansion constructed under `Assuming` can omit the hypothesis from its stored assumptions. A fourth issue is a source-proved transient-allocation hazard: a sparse rational-exponent series can allocate an enormous dense coefficient vector merely while constructing an optional compatibility property. A guarded large-denominator case was evaluated only after inserting the proposed protection. The accompanying code generates narrow source patches, defines regression tests, and records both native observations and independent exact computations. It does not claim that the full upstream test suite, or the complete supplied regression file, was run. Architectural and performance recommendations distinguish measured repository evidence, directly inspected implementation behavior, and proposed future work.

Contents

1	Executive assessment	4
1.1	What is genuinely valuable	4
1.2	The four immediate repairs	4
1.3	What should not be claimed	4
2	Revision, evidence, and reproducibility	5
2.1	Pinned input	5
2.2	Kinds of evidence	5
2.3	Inspection coverage	5
2.4	A repeatable local procedure	6
3	Architecture and data flow	6
3.1	The layered structure	6
3.2	Local coordinates	6
3.3	Representation variants	7
3.4	A maintainability concern: structural rather than nominal types	7
4	Mathematical invariants and the inversion engines	7
4.1	Complete blocks and exact weights	7
4.2	The coefficient formula	8
4.3	The three strategies are already meaningfully different	8
4.4	Precision transport is the real contract	9
4.5	Convergence versus asymptotic validity	9
4.6	Differentiation is deliberately more restrictive	9

5	Comparison with built-in Wolfram Language	10
5.1	The correct comparison baseline	10
5.2	Capability matrix	10
5.3	The output contract matters more than the function name	11
5.4	Native coverage probes	11
6	F01: a nested real-power guard can be bypassed	12
6.1	Reproduction and observed output	12
6.2	Root cause	13
6.3	Why this is not merely a loose error estimate	13
6.4	Repair	13
7	F02: sparse construction has an unbudgeted dense-export path	14
7.1	The allocation request	14
7.2	An important experimental correction	14
7.3	Repair in two stages	14
8	F03: native export forgets the logarithmic error factor	15
8.1	Reproduction	15
8.2	The analytical distinction	15
8.3	Repair choices	15
9	F04: ambient assumptions are used but not retained	16
9.1	Reproduction	16
9.2	Why the missing condition matters	16
9.3	Repair	16
10	F05: optimized inversion still pays for multi-index enumeration	17
10.1	The shared front end limits the benefit of separate engines	17
10.2	An exact counting example	17
10.3	Repair	17
11	Other API, UX, and maintainability findings	18
11.1	Cutoff and term-goal semantics are documented but difficult to predict	18
11.2	The Power observable changes meaning at a translated endpoint	18
11.3	Numerical application is not branch-aware verification	19
11.4	Ordinary arithmetic and held normalization have different guarantees	19
11.5	Result-head migration and persistence deserve a schema	19
11.6	Capability failures should be informative rather than accidental	19
11.7	Special-function dispatch should report the selected route	20

12 Performance analysis and concrete development opportunities	20
12.1 Separate four costs	20
12.2 A priority queue can replace repeated frontier sorting	20
12.3 Choose the engine using collision structure and coefficient cost	20
12.4 Polynomial caching is real, but a cache slot is not a byte budget	21
12.5 Canonicalization can be more expensive than jet arithmetic	21
12.6 The native importer has a linear integer-power path	21
12.7 Gamma/Barnes recurrence and refinement	21
12.8 What the existing benchmarks do and do not establish	22
13 Validation strategy and release engineering	22
13.1 Preserve the existing good provenance practices	22
13.2 Use a layered test portfolio	22
13.3 Property-based tests should generate admissible models	23
13.4 Resource tests need isolation	23
13.5 Packaging and compatibility improvements	23
14 Development roadmap	24
14.1 P0: close semantic and allocation defects	24
14.2 P1: make representation and resource contracts explicit	24
14.3 P2: extend verified functionality on the existing real scales	24
14.4 P3: broader scales without destroying the current invariants	24
14.5 A possible verification boundary	25
15 Using the accompanying artifacts	25
15.1 Independent mathematical checks	25
15.2 Native desired-contract regressions	25
15.3 Generate a canonical-source patch	25
16 Conclusion	26
A Source map and audit limits	26
B Compact experiment ledger	27

1 Executive assessment

1.1 What is genuinely valuable

The repository now implements considerably more than its name suggests. Besides local inversion with nonrational exponents and logarithmic coefficients, it exposes a generalized result object, arithmetic with transported uncertainty, coordinate changes, incremental inversion and refinement, finite exponential sectors, Fourier-valued coefficients, and selected special-function expansions. Its Gamma and Barnes inverse families retain a Lambert core rather than repeatedly expanding it away. The user guide explicitly describes these different scales and their different truncation conventions.[\[1, 2, 3, 10\]](#)

The central abstraction is worth retaining: an expansion is not just a finite expression. It also carries a coordinate, a remainder, a selected real approach, and sometimes a source or computation state from which additional coefficients can be obtained. This is precisely the information lost by treating every intermediate approximation as `Normal[Series[...]]`.

There is also evidence of unusually careful engineering. Sparse multiplication already prunes inadmissible products before multiplying coefficients. Integer jet powers use binary powering. The Newton implementation checks its symbolic residual after each refinement. The native special-function importer transports the `SeriesData` remainder rather than discarding it. The interval certificate code separates a numerical seed from exact proof arithmetic. Several tempting criticisms of an early prototype would therefore be wrong for this revision.[\[3, 7, 8, 9\]](#)

1.2 The four immediate repairs

ID	Priority	Finding	Evidence
F01	High	A common fractional-power helper accepts a pure remainder without proving a real branch; nested observables bypass the direct-operation guard.	Native reproduction and source path.
F02	High	Eager native-series conversion allocates a dense rational lattice without a size budget.	Exact source analysis; safe small case; guarded large case.
F03	High for analytical use	Native export drops the logarithmic degree of the error. The exported order is not a faithful numerical Big-O contract.	Native reproduction and elementary counterexample.
F04	High for reusable symbolic results	Ambient assumptions can be used during construction but absent from retained provenance.	Native reproduction under Assuming .

These are boundary and contract defects, not evidence that the Lagrange coefficient formula is wrong. The appropriate first response is a narrow hardening release with regression tests, not a rewrite of the entire algebra.

1.3 What should not be claimed

The package is not a complete implementation of general transseries, complex sectorial asymptotics, or uniform multivariate asymptotics. Conversely, Wolfram Language is not limited to

ordinary Taylor series: its native `Asymptotic` and `AsymptoticSolve` infer more general scales. A fair comparison is therefore between specific input classes, output contracts, and operations, not a blanket claim that one system can do asymptotics and the other cannot.[\[16, 17\]](#)

The repository’s own latest validation record is explicit: 305 tests passed in 18 selected files, but the full package suite was not run for the 1.8.0 milestone. This review preserves that qualification. Its successful native spot checks do not turn that partial coverage into a full-suite result.[\[13\]](#)

2 Revision, evidence, and reproducibility

2.1 Pinned input

The audited commit is 07a9781, in full 07a9781212beb2eeb9ff16aa625b50ac27974078. The repository’s root standalone distribution was downloaded and loaded in a native Wolfram evaluator. Its recorded size was 573,940 bytes and its SHA-256 was:

```
b2aebac8176649ff53634816f17e92c8f0ccb397f7606a5ac819b6f157b6d7ed
```

The native runtime reported:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
SystemID: Linux-x86-64
```

This satisfies the guide’s stated 15.0-or-later requirement, but it is not the same operating system and maintenance release as the repository’s recorded Wolfram 15.0.1 validation runs.[\[2, 13\]](#)

2.2 Kinds of evidence

Native observation means an evaluation completed and returned the result recorded in `results/native_observations.json`. **Source-proved** means the relevant control or allocation path is visible in the pinned code and its consequence follows without timing an extreme example. **Independent check** means the accompanying Python code checked a mathematical identity or count without running or emulating the package. **Recommendation** means a proposed change rather than a claim about an already observed failure.

Initial evaluator calls, and several later larger batches, returned HTTP 502. No functionality or performance conclusion is based on those failures. In particular, an attempted allocation trace produced no useful instrumentation. The dense-allocation finding is not presented as a measured memory benchmark. The unmodified package was never asked to allocate a billion-element vector.

2.3 Inspection coverage

The review reads the principal kernel and follows critical paths through series operations, automatic arithmetic, inverse-function expression handling, incremental inversion, exact interval certificates, native special-function import, and Gamma/Barnes inverse construction. It also reads the loading workflow, current validation record, user-guide entry contracts, the mathematical article’s convergence argument, and its calculus/error-envelope section. The source map in Appendix A identifies these inputs.

The larger family modules were surveyed through the main dispatch and guide, but not every line of every adapter or every archived report was re-audited. This matters particularly for

special-function endpoint coverage: a listed candidate head is not evidence that every order, parameter, or approach for that head is supported.

2.4 A repeatable local procedure

The archive contains two complementary mechanisms. `code/run_audit.wls` loads the exact standalone hash and evaluates the desired-contract regression file. With `-patched`, it applies the guard changes to a temporary copy before loading it. `code/harden_patch.py` instead generates a unified diff against the canonical modular source, and only edits that source when explicitly passed `-apply`.

The supplied regression file includes positive controls and tests expected to fail on the baseline. The full file was not executed during this review; individual native probes and the four guard behaviors were. Thus the archive is a reproducibility package, not a claim of comprehensive patch certification.

3 Architecture and data flow

3.1 The layered structure

A useful decomposition of the existing code is the following.[\[3, 4, 5, 6, 14\]](#)

Layer	Responsibility
Public syntax and branch admission	Held entry points, callable application, source/target conditions, inverse-function parsing and real branch selection.
Coordinate and family reduction	Finite endpoints and signed infinities; logarithmic and exponential reductions; exact Lambert, Gamma, Barnes, and other specialized forms.
Coefficient algebra	Sparse ordered weights, polynomial logarithmic coefficients, canonicalization, zero tests, truncated products, unit powers and compositions.
Computation strategy	Multi-index Lagrange inversion, grouped Lagrange inversion, precision-doubling Newton iteration, cached incremental frontiers.
Result and uncertainty	GeneralizedSeries , explicit error classes, exact carriers, branch/domain metadata, retained source, operation recipes and refinement statistics.
External boundaries	Native Series import/export, numeric evaluation, residual checks, exact certificates, formatting, standalone generation and loading.

Most of the serious issues found here arise in the last two layers. The mathematical core knows more than an optional export preserves, or a lower-level operation is callable without the branch checks enforced by a higher-level wrapper.

3.2 Local coordinates

At a finite endpoint the engine uses a positive displacement: $u = x - x_0$ from above, or $u = x_0 - x$ from below. At $+\infty$ it uses $u = 1/x$; at $-\infty$ it uses $u = -1/x$. All these choices reduce the limit

to $u \rightarrow 0^+$ and make the real logarithm unambiguous. The recorded direction is part of the result's meaning, not merely a display preference.[3]

This is an effective design for real asymptotics. It also explains several restrictions. A complex sector needs more information than a positive coordinate; two real sides are not interchangeable for fractional powers; and a target condition must be transported through an inverse rather than copied as though the source and target variables had the same role.

3.3 Representation variants

The simplest internal jet is a triple

$$(T, P, D), \quad T = \{(\beta_i, C_i(L))\}_{i=1}^n, \quad L = \log u,$$

representing

$$\sum_i u^{\beta_i} C_i(\log u) + \mathcal{O}\left(u^P (1 + |\log u|)^D\right). \quad (1)$$

$P = \infty$ denotes an exact finite expression. A more general result has an exact offset and carrier,

$$b(x) + c(x) \left[J(w(x)) + \mathcal{O}\left(w(x)^P (1 + |\log w(x)|)^D\right) \right]. \quad (2)$$

The absolute error contains $|c(x)|$. A cutoff on J is not automatically an absolute cutoff on the whole expression.[4, 12]

Other result kinds extend this representation: finite flat sectors, Fourier coefficient algebras, reciprocal-logarithmic Gamma/Barnes coefficients, and composite absolute-error envelopes. These are not merely alternate spellings for the same internal ring. In particular, a composite envelope can remain useful without having one canonical ordered coefficient list or a single scalar cutoff.[2, 5, 8, 12]

3.4 A maintainability concern: structural rather than nominal types

All these objects share `GeneralizedSeries[association]`. Dispatch repeatedly asks whether certain strings or keys are present, whether "Kind" equals a particular value, and whether a nested representation exists. That is flexible, but it moves type errors from construction time to later operations.

A stronger design would validate each representation against a schema and expose its capabilities explicitly. A result could declare whether it supports ordered addition, generic composition, derivative transport, refinement, native export, or certification. The current code already implements much of this distinction operationally; making it data would reduce duplicated conditionals and improve diagnostics. This is a refactoring recommendation, not a finding that the existing associations are intrinsically invalid.

4 Mathematical invariants and the inversion engines

4.1 Complete blocks and exact weights

A block is $u^\beta C(\log u)$ with all coefficients at that exact weight combined. Two monomials with different logarithmic degrees but the same β belong to one block. A `SeriesTermGoal` that counts blocks is therefore different from counting the summands after `Expand`.

With finitely many positive gaps $\delta_1, \dots, \delta_m$, the semigroup

$$S = \{k_1\delta_1 + \dots + k_m\delta_m : k_i \in \mathbb{N}\}$$

is locally finite even when the gaps are irrational: below a finite weight H , each $k_i < H/\delta_i$. This elementary observation justifies finite truncation. It does not make equality or ordering of arbitrary exact transcendental expressions decidable. The implementation's explicit failure on undecidable order is preferable to silently sorting by machine approximations.[3]

There are two separate normalization questions: whether two expressions denote the same real weight, and whether a polynomial coefficient vanishes under the active assumptions. Algebraic weights can benefit from **RootReduce**; transcendental identities may still require symbolic reasoning. Approximate comparisons should only be filters that are followed by a proof, not the source of the final ordering relation.

4.2 The coefficient formula

Consider the normalized finite model

$$v = au^p \left(1 + \sum_{i=1}^m u^{\delta_i} B_i(\log u) \right), \quad p \neq 0, \quad (3)$$

with a real and nonzero, and a selected real branch on which $z = (v/a)^{1/p} > 0$. Write the desired observable as u^r . For a multi-index k , set

$$n = |k|, \quad \sigma = k \cdot \delta, \quad Q_k(L) = \prod_i B_i(L)^{k_i}.$$

The implemented coefficient for $n \geq 1$ is

$$C_k^{(r)}(L) = \frac{(-1)^{nr}}{p^n \prod_i k_i!} \prod_{j=1}^{n-1} \left(\frac{d}{dL} + r + \sigma + pj \right) Q_k(L). \quad (4)$$

The $k = 0$ coefficient is 1. The expansion is $z^r \sum_k z^\sigma C_k^{(r)}(\log z)$, with equal weights aggregated. This is the core of **lagrangeCoefficient**; the incremental route applies the same Euler operators to cached polynomial products.[3, 7, 11]

For $f(u) = u + u^\alpha$ with $\alpha > 1$, formula (4) gives

$$u = y - y^\alpha + \alpha y^{2\alpha-1} - \frac{\alpha(3\alpha-1)}{2} y^{3\alpha-2} + \mathcal{O}(y^{4\alpha-3}). \quad (5)$$

At $\alpha = \sqrt{2}$, this matches the native package observation in Section 5.4. At $\alpha = 2$ the coefficients become signed Catalan numbers. The independent Python check composes the resulting polynomial with $u + u^2$ exactly through cutoff 14; all retained residual coefficients vanish.

4.3 The three strategies are already meaningfully different

The direct Lagrange route enumerates multi-indices and is an excellent independently understandable reference. The grouped route forms powers of the perturbation and combines terms at the same weight before applying the Euler operators. This can avoid repeated work when many indices collide. Newton instead solves the normalized implicit equation in the jet algebra and doubles a certified formal precision. The inspected implementation recomputes the residual after an update and fails rather than advancing an invalid precision.[7]

These alternatives should remain independently testable. Replacing every route with one shared implementation can make agreement tests vacuous: two entry points that eventually call the same buggy coefficient calculation are not independent checks.

4.4 Precision transport is the real contract

Suppose $F = A + \mathcal{O}(u^{P_1} M^{D_1})$ and $G = B + \mathcal{O}(u^{P_2} M^{D_2})$, where $M = 1 + |\log u|$. The product error follows from

$$FG - AB = A(G - B) + B(F - A) + (F - A)(G - B).$$

For nonzero leading blocks with valuations α_1, α_2 , the leading error exponent is bounded by

$$\min(P_1 + \alpha_2, P_2 + \alpha_1),$$

with the corresponding logarithmic degrees and any boundary products included. Exact zero and an empty jet with an unknown remainder must be distinguished. The package does distinguish them in its precision arithmetic.[3, 12]

Similarly, on a stable inverse branch with $f'(u) \asymp u^{p-1}$, an input perturbation $\mathcal{O}(u^\rho M^D)$ changes u^r by

$$\mathcal{O}(u^{\rho-p+r} M^D). \tag{6}$$

Converting to a positive target coordinate gives the package's inverse precision ceiling $(\rho - p + r)/|p|$, after accounting for the internal exponent used at infinity. This statement requires the branch/derivative stability hypothesis; a value bound alone cannot establish it for every arbitrary unknown function. The distinction between a finite formal model and an unspecified remainder family must remain visible in every certificate or residual result.[3, 9]

4.5 Convergence versus asymptotic validity

The article's finite-parameter convergence proof uses an analytic implicit equation near the normalized unknown $w = 1$. It replaces the small quantities $z^{\delta_i}(\log z)^j$ by independent parameters, applies the analytic implicit-function theorem, and then specializes them back. This is a sound structural explanation for convergence in that class. It is not a proof that every special-function expansion subsequently imported by the package converges.[11]

Gamma/Barnes forward tails are explicitly treated as finite-order Poincare information by the inverse code. The implementation stores a forward-model contract rather than declaring the infinite Stirling series convergent. This separation is correct and should survive future API consolidation.[10]

4.6 Differentiation is deliberately more restrictive

A magnitude estimate $R(u) = \mathcal{O}(u^P)$ does not imply $R'(u) = \mathcal{O}(u^{P-1})$. For example, $R(u) = u^P \sin(e^{1/u})$ has a derivative with an exponentially amplified oscillating part. The mathematical article and series-operation contract explicitly require derivative bounds for an unknown remainder.[12, 4]

This is not an unimplemented elementary feature to be “fixed” by blindly applying D to the remainder symbol. A useful extension is instead a separate source-differentiation route: differentiate a retained explicit source, re-expand it, and state that this is a new source computation rather than differentiation justified solely by the old error bound.

5 Comparison with built-in Wolfram Language

5.1 The correct comparison baseline

`Series` and `SeriesData` provide native truncated-series representations and arithmetic; `InverseSeries` performs reversion of appropriate native series, including examples with a nonlinear leading term. It is therefore incorrect to claim that native reversion universally requires a nonzero linear coefficient. `ComposeSeries` provides the corresponding native composition operation.[18, 19, 20, 21]

`Asymptotic` is a broader asymptotic front end with inferred scales, including scales beyond the standard Taylor/Laurent/Puiseux cases. Its documented scope includes expressions, integrals, transforms, and differential-equation results, with real or complex endpoints. The repository should be positioned as a specialized, inspectable representation and inversion layer, not as a substitute for all of that functionality.[16]

`AsymptoticSolve` is especially important: it already computes asymptotic solutions of implicit equations, can restrict to real solutions, and supports systems and multiple parameters. Its existence materially changes the claim from “Wolfram cannot invert asymptotically” to “these particular generalized inversions and retained error contracts are useful additions.”[17]

5.2 Capability matrix

Task	Built-in baseline	Audited package and assessment
Ordinary local power/Puiseux expansion	Native <code>Series</code> representation and operations.	Substantial overlap. Use native algorithms where their representation is adequate.
Reversion of native series	<code>InverseSeries</code> ; inverse variable can be selected.	Overlap for commensurable examples; package adds an explicit selected real chart and richer stored metadata.
Implicit asymptotic equations	<code>AsymptoticSolve</code> , including real restrictions and systems.	Specialized univariate real inversions with explicit coefficients, cut-offs and finite-model residuals.
Irrational power-log block algebra	<code>Asymptotic</code> is not limited to rational scales, but one native <code>SeriesData</code> lattice has rational step size.	Sparse arbitrary exact real weights and complete logarithmic blocks are a central strength.
Arithmetic and composition	Native series arithmetic and <code>ComposeSeries</code> .	Tracks several non-native error classes; not every result kind supports every operation.
Special-function asymptotics	Broad built-in knowledge used by <code>Series</code> and <code>Asymptotic</code> .	Adds carefully delimited adapters, exact carriers and remainder/provenance handling; much coverage is delegated.
Gamma/Barnes inverse organization	Native special functions and <code>ProductLog</code> are building blocks.	Dedicated retained-core reciprocal-logarithmic coefficient algebra and family-specific checks.

Task	Built-in baseline	Audited package and assessment
Real and complex geometry	Native asymptotic tools admit complex endpoints and directions where documented.	Positive real charts and selected real branches; general complex/Stokes geometry is outside the current contract.
Root verification	Symbolic and numerical solving are available, but are not the same object as this package's certificate.	Explicit rational interval certificates for supported stored expressions, with narrow, stated scope.
Refinement and provenance	Native series can be recomputed; representation and function interfaces differ.	Retained source, operation recipes and selected incremental states provide a useful additional workflow.

The native-column claims are grounded in the official function documentation; the package column follows the inspected implementation and guide.[\[16, 17, 18, 19, 20, 21, 2, 9\]](#)

5.3 The output contract matters more than the function name

An asymptotic expression, a formal truncated series, and a function approximation with a stated error class are different products. The package's opportunity lies in keeping the latter information composable. Native `AsymptoticLess` can help test strict asymptotic scale separation, but it is not by itself a numeric enclosure with constants and a validated interval.[\[22\]](#)

For common cases, the best architecture is hybrid. Let the native system provide exact simplification, Taylor coefficients, special-function expansions, and equation-solving primitives. Let the package own its sparse coefficient algebra, explicit branch records, and error-contract transformations. An adapter should import a native answer only after identifying which parts of that contract it actually supplies.

5.4 Native coverage probes

The following observations were obtained in Wolfram Language 15.0.0 on Linux. They are coverage probes, not speed comparisons. The exact calls are included in `code/comparison_probes.wl`.

For an ordinary inverse, both routes agree:

```
InverseSeries[Series[x + x^2, {x, 0, 4}], y]
(* SeriesData[y, 0, {1,-1,2,-5}, 1, 5, 1] *)

AsymptoticSolve[x + x^2 == y, x -> 0, y -> 0,
  Reals, SeriesTermGoal -> 4]
(* {{x -> y - y^2 + 2 y^3 - 5 y^4}} *)
```

The package's documented cutoff-5 inverse has the same four terms and an explicit `PowerLogRemainder[y,5,0]`.

For the irrational example, the direct native probe

```
AsymptoticSolve[x + x^Sqrt[2] == y,
  x -> 0, y -> 0, Reals, SeriesTermGoal -> 4]
```

returned unevaluated. The same syntax succeeds for the quadratic control above. Adding

Assumptions $\rightarrow y > 0$ did not resolve it and produced the documented-style warning that assumptions involving the problem variables are ignored. This is a result for these calls and this runtime, not a proof that no built-in transformation could solve the example.

The package produced

$$y - y^{\sqrt{2}} + \sqrt{2}y^{2\sqrt{2}-1} + \left(-3 + \frac{1}{\sqrt{2}}\right)y^{3\sqrt{2}-2}, \quad R = \mathcal{O}(y^{4\sqrt{2}-3}).$$

This is a concrete example of the repository's intended additional value, with coefficients independently explained by (4).

For `Erfc[x]` at infinity, the native `Asymptotic` call with `SeriesTermGoal` $\rightarrow 3$ returned

$$\frac{e^{-x^2}}{\sqrt{\pi}x} \left(1 - \frac{1}{2x^2}\right),$$

whereas the package call with the same option value returned

$$\frac{e^{-x^2}}{\sqrt{\pi}x} \left(1 - \frac{1}{2x^2} + \frac{3}{4x^4}\right), \quad R = \frac{e^{-x^2}}{x} \mathcal{O}(x^{-6}).$$

The coefficient knowledge is not unique to the package. The observation instead warns against assuming that equal option values identify equal truncations across interfaces. Comparison tests should normalize by the actual retained scale or target error, not just by a nominal term count.

6 F01: a nested real-power guard can be bypassed

6.1 Reproduction and observed output

Construct an expansion with no retained term but a nonzero error:

```
s = AsymptoticExpansion[-x^2, {x, 0, 1}];
{Normal[s], s["Remainder"]}
(* {0, PowerLogRemainder[x, 2, 0]} *)
```

The direct operation is guarded:

```
SeriesPower[s, 1/2]
(* Failure["UnknownLeadingTerm", ...] *)
```

But a nested observable bypasses that guard:

```
t = SeriesObservable[s, 1 + Sqrt[z], z];
{Head[t], Normal[t], t["Remainder"], t["TargetDomain"]}
(* {GeneralizedSeries, 1,
    PowerLogRemainder[x, 1, 0], x > 0} *)
```

All these outputs were reproduced natively.

An even stronger probe explicitly requires the square root to be real:

```
SeriesObservable[s,
  ConditionalExpression[1 + Sqrt[z],
    Element[Sqrt[z], Reals]], z]
```

It still returned $1 + \mathcal{O}(x)$ as a generalized expansion. On the actual source $z = -x^2$, $x > 0$, the required realness condition is false.

The ordinary forward entry also accepted

```
AsymptoticExpansion[Sqrt[Sin[x] - x], {x, 0, 1}]
```

with a generalized-series result whose finite expression was zero. Since $\sin x < x$ for sufficiently small positive x , this source has no real square-root branch on the admitted approach.

6.2 Root cause

The common `fwdPower` helper contains a shortcut for an empty retained jet. For a positive exponent, it transports the magnitude order by multiplying the power and rounding the logarithmic degree upward. It does so before establishing a real branch.[3]

The public `SeriesPower` route separately rejects a noninteger power of an uncertain empty jet. However, `seriesJetApply` calls `fwdPower` directly when it encounters a power inside a larger expression. The condition checker also uses this lower-level expression machinery; realness of an empty coefficient list can then be accepted vacuously.[4, 6]

This is a classic non-compositional validation error: an invariant enforced only at one public wrapper is not an invariant of the shared operation.

6.3 Why this is not merely a loose error estimate

For the unconditioned expression, the complex-valued function $1 + ix$ does satisfy a magnitude approximation $1 + \mathcal{O}(x)$. Thus it would be inaccurate to say that the magnitude estimate itself necessarily fails. The failure is that the package presents a real-branch calculation without the required proof, and the explicit `ConditionalExpression` probe accepts an impossible condition. The mathematical article expressly states the missing hypothesis for fractional powers of pure errors.[12]

6.4 Repair

Put the check in `fwdPower`, before its empty-jet shortcut:

```
If[T === {} && P != Infinity && !IntegerQ[rr],
  fail["UnknownLeadingTerm",
    "A pure remainder cannot establish the real branch " <>
    "of a noninteger power; refine the argument first."]];
```

An exact zero with $P = \infty$ remains distinguishable from an unknown error. Nonnegative integer powers retain their existing valid behavior. Where a retained explicit source can establish a sign, a higher layer may refine and retry; it must not manufacture that sign from a magnitude bound alone.

The shared guard rejected the nested observable in a native patched spot check. Regression coverage should include direct, nested, callable, conditional, composition, and automatically normalized forms, as well as exact zero and genuinely positive leading terms. Fixing only the one public example would leave equivalent paths open.

7 F02: sparse construction has an unbudgeted dense-export path

7.1 The allocation request

Both native-series conversion helpers construct a rational exponent lattice. In simplified notation they compute

```
den = LCM @@ Denominator /@ Append[exponents, remainderPower];
nmin = Min[exponents] den;
nmax = remainderPower den;
coeffs = Table[0, {nmax - nmin}];
```

and then place the sparse coefficients into this dense list. The real code handles the empty retained case separately, but it has no size guard on this allocation. Conversion is called during ordinary object construction, not only when a user later asks for the "SeriesData" property.[3]

Take

```
q = 1000000007;
AsymptoticExpansion[x^(1/q) + x,
  {x, 0, 1}, "MaxTerms" -> 32]
```

There is one retained block, at exponent $1/q$, and the first omitted block is at exponent 1. The converter requests

$$\text{den} = q, \quad n_{\min} = 1, \quad n_{\max} = q, \quad N = q - 1 = 1,000,000,006$$

zero entries. A resource limit on sparse terms does not constrain this quantity.

7.2 An important experimental correction

With $q = 101$, the native final **SeriesData** coefficient list had length *one*, not 100. The native constructor trims trailing zeros after receiving the input. That does not remove the preceding **Table** request. The finding concerns transient construction work, not the size of the final printed object.

No native peak-memory number is claimed, and no billion-slot baseline execution was attempted. The independent script verifies the allocation-count arithmetic without allocating a large list. After inserting the guard, the billion-denominator expression was evaluated natively and returned the correct sparse finite expression with a **Missing** native-export property.

7.3 Repair in two stages

The immediate repair is a preallocation bound:

```
If[nmax - nmin > 100000,
  Return[Missing["DenseRepresentationTooLarge",
    <|"RequestedSlots" -> nmax - nmin,
      "SlotLimit" -> 100000|>], Module]];
```

This does not discard the generalized expansion. It only declines an optional dense representation. The better API is a lazy export operation with a separate "MaxSeriesDataSlots" or byte budget. The representation should first estimate the lattice size and only allocate after the user

actually requests export. Rational denominators, not just large cutoffs, belong in the resource model. Pairwise coprime denominators can be more dangerous than a single conspicuously large exponent denominator.

The supplied patch implements the conservative constant cap in both converter sites. It does not pretend that a hardcoded cap is the final configurable-budget architecture.

8 F03: native export forgets the logarithmic error factor

8.1 Reproduction

The following native observation is decisive:

```
s = AsymptoticExpansion[x + x^2 Log[x], {x, 0, 2}];
{s["Remainder"], s["SeriesData"]}
(* {PowerLogRemainder[x, 2, 1],
    SeriesData[x, 0, {1}, 1, 2, 1]} *)
```

The package knows that the error is $\mathcal{O}(x^2(1 + |\log x|))$, but exports only the exponent cutoff 2.

8.2 The analytical distinction

For $x \rightarrow 0^+$,

$$\frac{|x^2 \log x|}{x^2} = |\log x| \rightarrow \infty.$$

Consequently $x^2 \log x$ is not $\mathcal{O}(x^2)$. At $x = e^{-n}$, the ratio is exactly n ; the independent check records this identity without floating-point underflow.

A native series order can be used as a formal exponent filtration, including contexts where coefficients themselves contain logarithms. That formal use is not the same as a faithful export of the package's stronger analytical error contract. The bug should therefore be described precisely: the compatibility property loses information needed to interpret its error as the original numerical Big-O class, rather than claiming that all formal uses of native series with logarithms are invalid.^[19]

The repository's native *importer* already recognizes this limitation. Its `SeriesData` branch explicitly loses half a lattice step to absorb unknown fixed logarithmic powers, instead of guessing the logarithmic degree. The exporter lacks a symmetric safeguard.^[8]

8.3 Repair choices

The safest default is to decline faithful native export when the remainder logarithmic degree is nonzero. The supplied patch returns `Missing["LogarithmicRemainderNotRepresentable"]` while preserving the original generalized object.

A richer export operation can offer three explicit modes. A lossless envelope returns the native finite part together with the original remainder and domain. A formal mode declares that only exponent-filtration information is being exported. A conservative analytical mode weakens P to $P - \varepsilon$, since

$$u^P(1 + |\log u|)^D = \mathcal{O}(u^{P-\varepsilon}) \quad \text{for each fixed } \varepsilon > 0.$$

In the last mode, ε must leave every retained exponent below the new cutoff, and the resulting rational lattice must still satisfy the export budget. Blindly subtracting $1/2$ can cross a retained exponent and invalidate the intended truncation.

Native export should also disclose that a bare `SeriesData` object does not carry the original selected branch or parameter-domain record. Remainder preservation and domain preservation are separate conversion obligations.

9 F04: ambient assumptions are used but not retained

9.1 Reproduction

In a fresh symbolic context:

```
s = Assuming[a > 0,
  AsymptoticInverse[a x + x^2, {x, 0}, {y, 3}]];
{Normal[s], s["Assumptions"]}
(* {y/a - y^2/a^3, True} *)
```

The native output succeeds under the ambient positivity assumption, but the returned association records `True`. The same expression with `Assumptions -> a > 0` explicitly supplied also succeeds.

`Assuming` works by extending the ambient assumptions used by functions that consult them. An expansion object intended for later reuse must retain every hypothesis that participated in its construction, regardless of whether that hypothesis arrived via an explicit option or ambient state.^[23]

9.2 Why the missing condition matters

The coefficients alone do not reveal the selected real branch. For $a > 0$, the source branch $x \rightarrow 0^+$ maps to positive small y . For $a < 0$, it maps to negative small y . An approximation generated using the first case cannot be presented as a parameter-unconditional result for the same selected approach.

The problem is particularly subtle because the initial expression looks reasonable. It can survive simplification, serialization, or transfer to a new kernel before the missing assumption becomes visible in an operation, a specialization, or a branch check. Keeping an expression but losing its proof hypotheses is not adequate computational provenance.

9.3 Repair

The public entry layer should choose and document one of two policies: combine explicit and ambient assumptions, or intentionally isolate itself from ambient assumptions. The first is more consistent with typical Wolfram usage. In either case, capture the effective assumptions exactly once, split parameter conditions from approach conditions, and propagate the resulting hypothesis record.

The supplied targeted patch changes the shared assumption splitter to use

```
clauses = With[{effective = ass && $Assumptions},
  If[Head[effective] === And,
    List @@ effective, {effective}]];
```

The native patched observation then stored $a > 0$ rather than `True`. This covers the inspected common entry path. A complete hardening pass should audit every independently exposed constructor and operation, not assume that all specialized paths necessarily pass through that splitter.

For maximum predictability, internal calculations can run with ambient assumptions reset after the captured hypotheses have been passed explicitly. This prevents a later unqualified simplification call from acquiring additional unstored context. Tests should construct an object inside `Assuming` or `Block`, inspect it outside, serialize/reload it, and compare it with a construction that supplied the same hypothesis explicitly.

10 F05: optimized inversion still pays for multi-index enumeration

10.1 The shared front end limits the benefit of separate engines

The main inverse constructor builds an `indexRegion` before dispatching fixed-cutoff work to `inverseBlocks`. The Newton branch of `inverseBlocks` does not need that region for its coefficient computation, but the constructor subsequently uses a multi-index frontier for the omitted block. In term-goal mode, the constructor first advances the incremental Lagrange state to determine the boundary, and can then recompute the result with Newton or grouped Lagrange.[\[3, 7\]](#)

Consequently selecting an optimized coefficient engine does not necessarily escape the combinatorial enumeration cost. It can encounter the index-state `MaxTerms` limit before benefiting from grouping or Newton iteration. This is a source-level performance and resource-architecture finding, not a native timing measurement; an attempted larger evaluation batch returned a connector error and supplies no additional evidence.

10.2 An exact counting example

For distinct gaps $1, 2, \dots, 8$, the number of nonnegative multi-indices of weight strictly below 19 is 1,313. There are only 19 possible integer weights, including zero. These counts follow from the truncated generating function

$$\prod_{j=1}^8 \frac{1}{1-t^j}.$$

They are directly relevant to a polynomial source $x + x^2 + \dots + x^9$ with inverse target cutoff 20, whose relative internal cutoff is 19. This is an enumeration count, not a claim that a particular low-budget Newton run was successfully benchmarked.

A separate depth illustration in the independent script has eight generators and depth 12. It contains $\binom{20}{8} = 125,970$ multi-indices but at most 97 distinct integer weights for gaps $1, \dots, 8$. The two examples use different truncation conventions; they must not be conflated.

10.3 Repair

Separate three questions: how to compute coefficients, how to discover the next admissible weight, and how to establish a valid omitted-term bound. Newton and grouped Lagrange should have method-native frontier logic or a shared frontier service based on *distinct weights*, without requiring all multi-index representations.

For commensurable weights, an integer lattice can make frontier discovery especially cheap. For general finitely generated positive weights, an exact priority queue with deduplication can enumerate distinct candidate weights. The coefficient engine can expand through one additional

complete boundary layer. If the boundary cancels, it can extend again, under an explicit frontier-search budget. A conservative valid bound is preferable to an unbounded search for the sharpest possible first omitted coefficient.

The direct multi-index route should remain available as the reference and for coefficient-by-index queries. Removing unnecessary enumeration from other strategies is an architectural optimization, not an argument for deleting it.

11 Other API, UX, and maintainability findings

11.1 Cutoff and term-goal semantics are documented but difficult to predict

The public guide carefully documents an exclusive cutoff in the ordinary coordinate, a relative cutoff inside Gamma/Barnes and exponential carriers, per-carrier conventions for some native special-function results, and inherited inverse conventions for direct inverse-function expressions. These are real mathematical differences, but one argument position currently conceals them.[2]

A result should expose a compact **"TruncationDescriptor"** containing the coordinate, whether the cutoff is absolute or carrier-relative, which blocks were counted, the requested target, and the achieved precision. A display or query such as

```
SeriesInformation[s, "Truncation"]
(* <|"Coordinate" -> 1/X, "RelativeTo" -> carrier,
    "Cutoff" -> 5, "CountUnit" -> "CompleteBlock"|> *)
```

would make the convention visible without requiring inspection of a large association. This is a proposed interface, not an existing public symbol.

The forward rule form also differs from native **Asymptotic**'s default-leading-term idiom: the documented package usage supplies a cutoff or positive term goal. A leading-block default would be convenient, but should be introduced deliberately rather than by weakening validation accidentally.[2, 16]

11.2 The Power observable changes meaning at a translated endpoint

For a finite source endpoint and $r \neq 1$, **"Power"** $\rightarrow$ **r** means $(x - x_0)^r$; for $r = 1$, it means the original source variable including its translation. At infinity it means x^r . The guide explicitly states this, and the native probe

```
Normal[AsymptoticInverse[x, {x, 1}, {y, 3}, "Power" -> 2]]
(* (y - 1)^2 *)
```

confirms the documented behavior. It is therefore not reported as a mathematical bug.[2, 3]

It is nevertheless surprising: a user asking for the square of an inverse may expect y^2 . An explicit local-power option or an observable function would eliminate the ambiguity. A migration can preserve **"Power"** while introducing a clearly named **"LocalPower"** and a true source observable. The observable's identity should be retained so residual and numerical checks know which equation to verify.

11.3 Numerical application is not branch-aware verification

The object's numeric application rule substitutes a numerical value into the finite expression. It does not itself enforce an asymptotic threshold, verify the stored domain, or return a numeric error bound. This can be a useful convenience, but it should remain visibly distinct from `InverseNumericalCheck` and `InverseCertificate`.[\[3, 9\]](#)

A future evaluator could return the approximation together with domain-check status, a scale estimate, and any available certificate. A scale estimate is not an error bound with known constant. The raw finite-expression route should remain available through `Normal`, so expert users are not forced into expensive verification for every plotting call.

11.4 Ordinary arithmetic and held normalization have different guarantees

The implementation already acknowledges that native evaluation can simplify an expression before an upvalue has a chance to validate its denominator. `SeriesNormalize` is held specifically to inspect an arithmetic expression before that cancellation. This is a good existing safeguard, not an omitted feature discovered by this review.[\[5\]](#)

Documentation and tests should emphasize the difference between correlated repeated use of one approximation and independent uncertain approximations with the same finite expression. The identity of unknown errors affects cancellation. Treating all errors as independent is conservative; canceling them requires an actual identity or common-source relation. An eventual provenance DAG could make this distinction explicit, but it should not infer correlation merely from equal printed expressions.

11.5 Result-head migration and persistence deserve a schema

Version 1.8.0 renames the public head to `GeneralizedSeries` and does not export the old head as an alias. The validation record documents this migration. Saved patterns, package integrations and serialized expressions can nevertheless break even though `StandardForm` output looks unchanged.[\[13\]](#)

Add an object schema version and a non-evaluating migration path for supported old data. A temporary compatibility alias may or may not be desirable, but persisted objects need a policy separate from symbol renaming. Refinement recipes should state the package/model version they rely on. A future object reader should validate fields before executing any retained recipe.

11.6 Capability failures should be informative rather than accidental

Gamma/Barnes inverse results deliberately support a narrower set of generic operations because their coefficient variable is reciprocal-logarithmic rather than a polynomial in the ordinary log coordinate. This is a legitimate mathematical restriction. Similar limits apply to depth-truncated symbolic exponents and composite envelopes.[\[4, 10, 5\]](#)

An operation matrix should be available programmatically. Failure data can then report the unsupported capability, the current scale, and a valid alternative operation. The coefficient-query probe on a forward object returned a generic `InvalidArguments` failure rather than a crash; better diagnostics could distinguish a syntactically valid but semantically inappropriate result kind from a malformed call.

11.7 Special-function dispatch should report the selected route

The candidate head lists are broad, but a successful expansion still depends on real-domain proofs, parameter regimes, supported native tree shapes, coefficient algebras, and available asymptotic orders. An adapter list is not an endpoint-coverage table.[8, 2]

Expose a diagnostic route trace: identity normalization, native expansion request, imported scale, branch proof, and rejection reason. This would be especially valuable when mathematically equivalent expressions choose different adapters. The trace should be optional and structured, so normal output remains compact and tests can assert the chosen route without parsing English messages.

12 Performance analysis and concrete development opportunities

12.1 Separate four costs

A realistic cost model distinguishes support enumeration, coefficient arithmetic, symbolic proof/normalization, and representation conversion. Counting output blocks alone hides all four. A ten-block answer can require thousands of colliding indices, high-degree log polynomials, difficult exact comparisons, or a huge dense export lattice.

The current "MaxTerms" is used in several ways: limits on indices and frontier states, retained products, expression leaf counts, and native polynomial powers. These are not interchangeable resource measures. Supplement it with independent budgets for index states, candidate products, coefficient degree/size, exact-integer bit length, native export slots, solver time, and overall wall-clock or memory limits. Report which budget was exhausted and what progress was retained.[3, 7, 8, 5, 10]

12.2 A priority queue can replace repeated frontier sorting

The incremental state uses stringified multi-indices as association keys, scans a boundary for all entries at the next weight, and sorts boundary weights to find the following layer. This is clear and correct-looking, but repeated full scans and sorts can dominate a long refinement.[7]

An exact min-priority queue can reduce frontier-maintenance work, provided equal weights are extracted as one complete layer. A separate canonical key is needed for deduplication; a display string should not become an implicit algebraic identity format. For small frontiers the existing association may remain faster, so a thresholded implementation and measured crossover are preferable to unconditional replacement.

12.3 Choose the engine using collision structure and coefficient cost

The default method is Lagrange, and the code already offers explicit Newton and grouped alternatives. A proposed `Method -> Automatic` should use an estimated cost rather than simply select the newest implementation. Useful features include the number of generators, anticipated index count, estimated number of distinct weights, common rational lattice, log-polynomial degrees, and available cached state.

For dense integer supports, Newton with a fast lattice backend can be attractive. For many collisions and modest polynomial degrees, grouped Lagrange can win. For a few requested coefficients or a small sparse support, the direct formula can be simplest and cheapest. For

term-goal queries, avoid paying for a complete Lagrange expansion just to select the cutoff for another engine, as discussed in F05.

12.4 Polynomial caching is real, but a cache slot is not a byte budget

The incremental implementation already caches nonconstant log-polynomial powers and trims optional cache entries when frontier growth needs the same budget. It explicitly treats a full cache as a performance condition rather than an input failure. This is a strong design choice.[7]

The remaining issue is size accounting. One cached polynomial can be much larger than another. Memory-aware eviction should consider coefficient count, degree, exact-number size, and recomputation cost. The same observation applies to whole retained computation states and nested operation recipes. Copy-on-write behavior may reduce some duplication in one kernel, but serialization and independently constructed subtrees still need explicit bounds.

12.5 Canonicalization can be more expensive than jet arithmetic

The shared coefficient normalization expands expressions, rationally combines them, simplifies algebraic numbers, and rewrites logarithms of rational/integer constants using factorization. These are mathematically useful canonicalizations, but integer factorization and unrestricted symbolic simplification can dominate otherwise small calculations.[3]

A staged policy would normalize cheap structural identities first, use cached algebraic canonical forms next, and defer expensive transformations until equality or zero testing actually requires them. Parameter assumptions belong in cache keys. Expose separate normalization budgets and return an unresolved proof obligation instead of silently replacing an exact predicate with floating-point evidence.

This is a source-derived latency risk, not a demonstrated regression on a specific semiprime. The benchmark suite should include adversarial constants and large exact coefficient heights to establish whether it matters in actual workloads.

12.6 The native importer has a linear integer-power path

The main jet algebra already uses binary powering, but the inspected native-tree importer handles a nonnegative integer power with a linear fold over that exponent.[3, 8]

Repeated squaring of value/error pairs can reduce the number of multiplication stages. Its envelope must still account for all propagated uncertainty; replacing the fold with ordinary expression exponentiation and dropping the error is not a valid optimization. Also measure envelope expression growth: fewer multiplication stages do not guarantee smaller symbolic bounds without careful normalization.

12.7 Gamma/Barnes recurrence and refinement

The Gamma inverse coefficient routine builds a unit correction order by order, repeatedly expands a phase with `Series`, and extracts the next coefficient. Some frontier searches recompute rows from the beginning at increased order. This is transparent but offers opportunities for retained recurrence state and truncated polynomial arithmetic.[10]

The important mathematical structure should be preserved. For a logarithmic target Y , the leading Gamma core is

$$X = \frac{Y}{W(Y/e)}, \quad X \log X - X = Y.$$

The Barnes core uses

$$X = \sqrt{\frac{4Y}{W(4Y/e^3)}}, \quad \frac{1}{2}X^2 \log X - \frac{3}{4}X^2 = Y,$$

with the source offset and affine transformations accounted for separately. The code keeps complete polynomials in $1/\log X$ for Gamma and $1/(\log X - 1)$ for Barnes. Flattening these into one indiscriminate power-log list would erase the reason that the ordering is manageable.[10]

12.8 What the existing benchmarks do and do not establish

The repository records the following local 1.8.0 comparison against an immutable baseline, with one warm-up and three measured runs in fresh Wolfram 15.0.1 kernels. These values are repository evidence, not rerun measurements from this audit.[13]

Fixture	Before (s)	After (s)
100 translated-coordinate inversions	0.12115	0.00418
Fourier merge with repeated frequencies	0.19112	0.09396
200-by-200 Fourier product below weight 4	1.28634	0.00776
Public Fourier inverse through target weight 4	0.02514	0.02325

The first three are substantial local improvements. The public inverse shows a much smaller change, illustrating why helper microbenchmarks should not be advertised as end-to-end speedups. Earlier records also include essentially unchanged public irrational/Gamma/Barnes examples and a noisy native-tail case whose follow-up did not establish a consistent regression. The validation documentation handles this distinction responsibly.[13]

A future benchmark harness should record peak memory as well as time, isolate cold startup from warm algebra, retain the full output/remainder for equivalence checks, and compare matched mathematical precision. Cross-method tests should also record enumeration work, coefficient evaluations, cache hits, and solver time, so an improvement can be attributed to the correct layer.

13 Validation strategy and release engineering

13.1 Preserve the existing good provenance practices

The standalone distribution is generated from canonical modules in a recorded order, with source hashes and deterministic line-ending normalization. The builder checks dependency structure and has focused Python tests. The inspected GitHub Actions workflow checks standalone freshness and runs those builder tests. It does not itself run the native mathematical suite.[14, 15]

The latest validation record distinguishes focused test selections, source hashes before and after execution, isolated loading checks, and unrun full suites. Keep these records immutable and tied to the exact revision. A green builder job means the distribution is fresh; it does not mean the mathematical implementation is validated.

13.2 Use a layered test portfolio

A practical portfolio has four layers. Small deterministic contract tests catch F01–F04 and representation invariants. Differential tests compare independent internal engines and native facilities

on their common domain. Metamorphic tests exercise identities and coordinate transformations. A slower integration suite covers branch families, special-function parameters, refinement histories, serialization, and resource behavior.

For the algebra, test permutation invariance of summands, equivalent algebraic exponents, complete collision cancellation, exact zero versus uncertain empty jets, and monotonicity of transported precision. For inverses, test $f(g(y)) - y$ and also branch membership; a formal residual alone does not distinguish every inverse branch. For refinement, compare both finite expressions and retained uncertainty with direct higher-order construction.

For parameter handling, test all permitted sign regimes, zero-boundary transitions, contradictory assumptions, ambient and explicit hypotheses, and leaving the dynamic scope that supplied the assumptions. For native import/export, test logarithmic tails, large rational denominators, exact exponentials, oscillatory coefficients, and round trips that must be rejected rather than silently weakened.

13.3 Property-based tests should generate admissible models

Generate finite exact models of the form (3) with positive rational or algebraic gaps and log polynomials of bounded degree. The generator should know the selected branch and the assumptions it supplied. It should produce both admissible inputs and deliberately inadmissible near-neighbors, rather than treating every failure as a bug.

For commensurable supports, compare with a common uniformizing variable and native series reversion. For irrational supports, verify the independent coefficient formula and model residual. For negative tests, erase a leading block, move a domain condition across a branch boundary, or replace a known sign by an unknown parameter. The expected result is often a structured failure, not another expansion.

13.4 Resource tests need isolation

An out-of-memory process cannot reliably export a test report afterward. Large-denominator and explosive-polynomial tests should run in isolated kernels with external resource limits. Estimate the expected allocation before launching the kernel. The billion-denominator case in this archive is guarded so it cannot accidentally run on the baseline through the provided regression driver.

Keep correctness tests independent of wall-clock thresholds where possible. A test that asserts a specific failure category and budget counter is less flaky than one that assumes all machines finish a symbolic simplification in a fixed fraction of a second. Timing regressions belong in a separately interpreted benchmark track.

13.5 Packaging and compatibility improvements

The download-then-load workaround is documented and supported by repository loading tests; replacing it with an untested direct HTTP loader would regress reliability. For reproducible work, the public quick-start should make a pinned release or commit at least as visible as the moving `main` URL.^[14, 2]

A release matrix should distinguish the minimum supported Wolfram version from versions actually tested. Native `Series` output shapes and special-function behavior are part of the effective dependency surface. Maintain a small import-boundary fixture set for each supported release and platform, and record when a returned tree shape is intentionally rejected.

14 Development roadmap

14.1 P0: close semantic and allocation defects

Apply the four guard changes, rebuild the standalone file from the canonical modules, and run the existing affected suites plus the new negative cases. Preserve failure categories where practical, but prioritize refusing an unproved branch over maintaining previously accepted invalid behavior. Extend the assumption-capture audit to independently exposed constructors. Add faithful-versus-formal native-export semantics to the guide.

The acceptance criterion is not merely that the four examples change output. Equivalent nested entry paths must obey the same contract, ordinary valid inversions must remain unchanged, and large sparse objects must remain constructible without optional dense export.

14.2 P1: make representation and resource contracts explicit

Introduce object schema versions, capability queries, a truncation descriptor, and uniform budget diagnostics. Make native export lazy. Freeze proof hypotheses and source identity in refinement recipes. Keep error magnitude, differentiability information, and numerical certification as distinct fields rather than overloading one `Remainder` property.

At the strategy layer, remove mandatory multi-index enumeration from methods that do not require it. Measure the resulting end-to-end gain before adding further micro-optimizations. Implement algorithm selection only after collecting representative crossovers rather than inventing a universal threshold.

14.3 P2: extend verified functionality on the existing real scales

Several useful extensions fit the present architecture. A source-differentiation/re-expansion operation can complement derivative-contract calculus. Integration can be added with explicit integration constants and remainder hypotheses; a logarithmic resonant exponent -1 needs special treatment. A native-adapter protocol can make supported endpoint/parameter conditions auditable. Certificate arithmetic can be extended with rigorous enclosures for additional functions, one family at a time.

The existing convergence section also gives an explicit conditional majorant based on a marker-disc argument. Making that theorem available as a separate numerical tail-bound operation would complement interval root certificates: it would bound the truncation tail under a checked smallness condition, rather than merely certify a root near one approximation. Such a feature should report the verified majorant condition and the exact scope of the bound.[\[11, 9\]](#)

Gamma/Barnes certificate support is another natural target, but it requires rigorous function/derivative enclosures or explicit finite-order remainder inequalities. High working precision alone is not a replacement for those ingredients.

14.4 P3: broader scales without destroying the current invariants

Uniform parameter asymptotics, coalescing branches, general complex sectors, Stokes switching and unrestricted transseries are larger projects. They require explicit scale families and branch geometry, not merely more cases in a generic `Switch`. Near a vanishing leading coefficient, the dominant balance can change. Near coincident exponents, block order can change. Near a critical inverse point, one may need a new uniformizer and multiple branch labels.

A sensible route is to introduce these as new representation types with independently stated algebraic and error contracts. The present real power–log implementation can remain a well-tested subsystem. There is no need to weaken its existing meaning to advertise a larger nominal scope.

14.5 A possible verification boundary

The most promising small verification target is not an entire computer algebra system. It is the bookkeeping kernel: ordered finite supports, complete-block merging, error-exponent transport, interval containment, and replayable certificate checks. Symbolic simplification can supply candidate identities accompanied by small checkable witnesses where feasible.

For numerical certificates, export the rational endpoints, arithmetic operations, derivative bound and residual enclosure as a proof object with a small independent checker. For formal inverses, export a truncated implicit-equation residual in a declared coefficient ring. Both approaches isolate trusted arithmetic from large heuristic discovery procedures. This is a development proposal, not a claim that the current package is formally verified.

15 Using the accompanying artifacts

15.1 Independent mathematical checks

Run:

```
python code/verify_math.py --output results/independent_checks.json
python -m unittest discover -s code -p 'test_*.py' -v
```

The Python calculation does not implement Wolfram evaluation semantics. It verifies exact allocation counts, the logarithmic-error counterexample, a Catalan reversion identity, and collision-count examples. Its test output is included separately from the native observations.

15.2 Native desired-contract regressions

In a fresh Wolfram kernel, run either:

```
wolframscript -file code/run_audit.wls
wolframscript -file code/run_audit.wls --patched
```

The first command is expected to expose baseline defects; a nonzero exit status is intentional when desired-contract tests fail. The second validates anchors and patches a temporary standalone copy. It does not alter an installed package or the GitHub repository. Both require access to the pinned source URL unless `ASYMPTOTIC_AUDIT_PACKAGE` points to a local copy with the exact audited hash.

The complete regression file has not been run in this session. The native observations and patched spot checks recorded in the archive are narrower. This distinction also applies to the positive certificate/refinement controls supplied for future execution.

15.3 Generate a canonical-source patch

From a local checkout of the audited revision:

```
python /path/to/audit/code/harden_patch.py /path/to/Asymptotic
```

```
# Prints a unified diff; does not modify the checkout.

python /path/to/audit/code/harden_patch.py /path/to/Asymptotic --apply
# Validates all anchors, writes a non-overwriting backup, then edits.

cd /path/to/Asymptotic
python validation/build_standalone.py
python validation/build_standalone.py --check
python -m unittest discover -s validation -p test_standalone.py -v
```

Then run the applicable native suites and documentation checks before committing. The patch generator deliberately refuses a source whose anchors no longer match. It does not attempt a fuzzy merge into an unknown future version.

The shipped guard patch adds descriptive diagnostics to the same guard logic tested in the native spot check. It is a minimal hardening proposal: the dense-export cap is fixed rather than a public option, native export remains eager but bounded, and assumption capture still needs a complete audit of all specialized public entries.

16 Conclusion

The repository has a coherent mathematical core and a useful software direction: preserve asymptotic structure, selected real branches and uncertainty as first-class data. Native evaluation confirms that its irrational inverse example produces a useful result where the straightforward `AsymptoticSolve` probe remains unevaluated on the tested runtime. Its special-function layer should be understood as a precision-aware integration with native knowledge, not a wholesale replacement for it.

The highest-value next work is correctness at the boundaries. Shared operations must enforce the hypotheses their formulas require. Compatibility exports must not silently strengthen error claims or allocate an enormous lattice. Assumptions used to select a branch must remain attached to the result after the dynamic scope ends. Once those issues are closed, method-native frontier discovery, clearer representation capabilities and better resource accounting provide a technically grounded path to broader and faster functionality.

A Source map and audit limits

The following paths are pinned to the revision on the title page. A named helper is a more stable locator than a line number in the generated standalone, where module inclusion changes offsets.

Source or group	Reviewed role
<code>AsymptoticInverse/Kernel/AsymptoticInverse.wl</code>	Public contracts; exact ordering; sparse and precision jets; forward expansion; three inversion dispatches; native export; formatting; residual and coefficient access.
<code>AsymptoticInverse/Kernel/SeriesOperations.wl</code>	Carrier representation; ordinary operations; nested observables; composition; derivative/refinement contracts and replay.
<code>AsymptoticInverse/Kernel/SeriesArithmetic.wl</code>	Automatic arithmetic, held normalization, regular operands and fallback decisions.

Source or group	Reviewed role
<code>AsymptoticInverse/Kernel/InverseFunctionExpressions.wl</code>	Held public entry flow, condition-on-jet checking, native inverse identity recovery and branch provenance.
<code>AsymptoticInverse/Kernel/IncrementalInverse.wl</code>	Frontier state; polynomial cache; Newton doubling and residual verification; grouped inversion.
<code>AsymptoticInverse/Kernel/NativeSpecialFunctions.wl</code>	Initial importer/tree algebra, magnitude majorants, native tail semantics and coefficient restrictions.
<code>AsymptoticInverse/Kernel/InverseCertificates.wl</code>	Exact rational enclosures, derivative separation, domain checks and root-existence/error logic.
<code>AsymptoticInverse/Kernel/GammaInverse.wl</code>	Gamma/Barnes model admission, retained cores, coefficient recursion and finite-order contracts.
<code>AsymptoticInverse/Documentation/UserGuide.md</code>	Public usage, loading, function overview and initial option contracts; broader families surveyed through this overview and dispatch.
<code>validation/README.md</code>	Latest focused evidence and preceding refactor/loading records.
<code>docs/development/README.md</code> ; <code>.github/workflows/standalone.yml</code>	Standalone provenance, deterministic build and inspected CI job.
<code>article/sections/07-convergence.tex</code> ; <code>article/sections/17-calculus.tex</code>	Finite-parameter convergence, explicit majorant, error calculus and operation hypotheses.

This is not a claim of exhaustive line-by-line inspection of all 38 canonical kernel sources, every test, all archived reports, or every special-function branch. The prioritized findings are grounded in the critical paths identified here.

B Compact experiment ledger

Experiment	Status and result
Pinned native load	Succeeded; runtime, byte count and SHA-256 recorded.
Direct versus nested fractional power	Reproduced direct rejection and nested acceptance on a negative-source pure remainder.
Conditional realness probe	Reproduced acceptance despite an impossible condition on the actual source.
Forward square root of $\sin x - x$	Reproduced a generalized result instead of real-domain rejection.
Logarithmic native export	Reproduced loss of remainder log degree.
Ambient Assuming	Reproduced successful construction with stored assumptions True .

Experiment	Status and result
Rational inverse native controls	Native <code>InverseSeries</code> and <code>AsymptoticSolve</code> returned the expected quadratic inverse.
Irrational inverse comparison	Package returned four blocks; the direct native solver call stayed unevaluated on the tested version.
<code>Erfc</code> comparison	Both returned exponential asymptotics; nominal term-goal values did not yield equal retained terms.
Four guard logic checks	Succeeded for nested rejection, conservative native export, bounded large-denominator construction and captured hypotheses.
Full upstream suite	Not run.
Complete supplied regression file	Not run; individual observations do not imply a complete pass.
Native timing/memory benchmark	Not established. Failed/empty trace attempts were excluded from conclusions.

References

- [1] V. Reshetnikov, *Asymptotic repository README*, audited revision. [README.md](#).
- [2] V. Reshetnikov, *AsymptoticInverse User Guide*, audited revision. [AsymptoticInverse/Documentation/UserGuide.md](#).
- [3] V. Reshetnikov, *Canonical main kernel*, audited revision. [AsymptoticInverse/Kernel/AsymptoticInverse.wl](#). Especially `fwdPower`, `splitApproachInput`, `makeSeriesData`, `makeInverseSeriesData`, and the inverse-construction dispatch.
- [4] V. Reshetnikov, *Explicit series calculus*. [AsymptoticInverse/Kernel/SeriesOperations.wl](#).
- [5] V. Reshetnikov, *Automatic series arithmetic*. [AsymptoticInverse/Kernel/SeriesArithmetic.wl](#).
- [6] V. Reshetnikov, *Inverse-function expression handling*. [AsymptoticInverse/Kernel/InverseFunctionExpressions.wl](#).
- [7] V. Reshetnikov, *Incremental and grouped inversion*. [AsymptoticInverse/Kernel/IncrementalInverse.wl](#).
- [8] V. Reshetnikov, *Structured native special-function importer*. [AsymptoticInverse/Kernel/NativeSpecialFunctions.wl](#).
- [9] V. Reshetnikov, *Exact rational inverse certificates*. [AsymptoticInverse/Kernel/InverseCertificates.wl](#).
- [10] V. Reshetnikov, *Gamma/Barnes inverse construction*. [AsymptoticInverse/Kernel/GammaInverse.wl](#).
- [11] V. Reshetnikov, *Real asymptotic expansions and inverse functions: Convergence*. [article/sections/07-convergence.tex](#).
- [12] V. Reshetnikov, *Calculus of expansions with transported precision*. [article/sections/17-calculus.tex](#).
- [13] V. Reshetnikov, *Review and validation record*, especially the 1.8.0 and shared-arithmetic-refactor milestones. [validation/README.md](#).
- [14] V. Reshetnikov, *Development and provenance*. [docs/development/README.md](#).
- [15] V. Reshetnikov, *Standalone package freshness workflow*. [.github/workflows/standalone.yml](#).
- [16] Wolfram Research, *Asymptotic*, Wolfram Language documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [17] Wolfram Research, *AsymptoticSolve*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [18] Wolfram Research, *Series*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [19] Wolfram Research, *SeriesData*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.

- [20] Wolfram Research, *InverseSeries*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [21] Wolfram Research, *ComposeSeries*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [22] Wolfram Research, *AsymptoticLess*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticLess.html>.
- [23] Wolfram Research, *Assuming*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Assuming.html>.